

## Assignment 5: OOP

1. Write a program to maintain student information. For each student store studid, name, m1, m2 and m3 (marks of 3 subjects ). Accept information for 2 students and display it as follows.

**Student Details:**

---

**Student Id**  
**Name:** Divya  
**M1 :** 78  
**M2:** 86  
**M3:** 89

class student:

```
def __init__(self, sid=0, sname="", m1=0, m2=0, m3=0):
    print("Student Constructor Called!")

    self.__sid = sid
    self.__sname = sname
    self.__m1 = m1
    self.__m2 = m2
    self.__m3 = m3

def __str__(self):
    return f'Sid : {self.__sid}, Sname : {self.__sname}, M1 : {self.__m1}, M2 : {self.__m2}, M3 : {self.__m3}'

# Setters
def set_sid(self, sid):
    self.__sid = sid

def set_sname(self, sname):
    self.__sname = sname

def set_m1(self, m1):
    self.__m1 = m1

def set_m2(self, m2):
    self.__m2 = m2

def set_m3(self, m3):
    self.__m3 = m3

# Getters
def get_sid(self):
    return self.__sid

def get_sname(self):
    return self.__sname

def get_m1(self):
```

```
    return self.__m1
```

```
def get_m2(self):  
    return self.__m2
```

```
def get_m3(self):  
    return self.__m3
```

```
Student = {}
```

```
def addStudent():
```

```
    id = int(input("Enter id: "))  
    name = input("Enter name: ")  
    m1 = int(input("Enter Marks m1: "))  
    m2 = int(input("Enter Marks m2: "))  
    m3 = int(input("Enter Marks m3: "))
```

```
    s = student(id, name, m1, m2, m3)
```

```
    Student[id] = s
```

```
    print(f"Added student: {s}")
```

```
    return True
```

```
def displayStudent():
```

```
    print("Student Details:")
```

```
    for sid, s in Student.items():
```

```
        print(f"Student ID : {s.get_sid()}")  
        print(f"Name : {s.get_sname()}")  
        print(f"M1 : {s.get_m1()}")  
        print(f"M2 : {s.get_m2()}")  
        print(f"M3 : {s.get_m3()}")  
        print("-----")
```

```
    return True
```

```
choice = -1
```

```
while choice != 0:
```

```
    choice = int(input("""  
Enter Choice:  
1. Add Student
```

2. Display All Students

0. Exit

"""))

match choice:

case 1:

status = addStudent()

print("Successfully Added") if status else print("Unsuccessful")

case 2:

displayStudent() if Student else print("No Students!")

case 0:

print("Thanks")

case \_:

print("Wrong Choice")

---

**2. Write a menu driven program to maintain student information. Modify Student class created in previous assignment. Add a member gpa in student class, add a function in student class to return GPA of a student**

**calculateGPA()**

**$\text{gpa} = (1/3)*m1 + (1/2)*m2 + (1/4)*m3$**

**Create an array to store Multiple students.**

**1. Display All Student**

**2. Search by id**

**3. Search by name**

**4. calculate GPA of a student**

**5. Exit**

class student:

def \_\_init\_\_(self, sid=0, sname="", m1=0, m2=0, m3=0):

print("Student Constructor Called!")

self.\_\_sid = sid

self.\_\_sname = sname

self.\_\_m1 = m1

self.\_\_m2 = m2

self.\_\_m3 = m3

def \_\_str\_\_(self):

return f'Sid : {self.\_\_sid}, Sname : {self.\_\_sname}, M1 : {self.\_\_m1}, M2 : {self.\_\_m2}, M3 : {self.\_\_m3}'

# Setters

def set\_sid(self, sid):

self.\_\_sid = sid

```

def set_sname(self, sname):
    self.__sname = sname

def set_m1(self, m1):
    self.__m1 = m1

def set_m2(self, m2):
    self.__m2 = m2

def set_m3(self, m3):
    self.__m3 = m3

# Getters
def get_sid(self):
    return self.__sid

def get_sname(self):
    return self.__sname

def get_m1(self):
    return self.__m1

def get_m2(self):
    return self.__m2

def get_m3(self):
    return self.__m3

# GPA
def calculateGPA(self):
    return (1/3) * self.__m1 + (1/2) * self.__m2 + (1/4) * self.__m3

```

```
Student = {}
```

```

def addStudent():

    id = int(input("Enter id: "))
    name = input("Enter name: ")
    m1 = int(input("Enter Marks m1: "))
    m2 = int(input("Enter Marks m2: "))
    m3 = int(input("Enter Marks m3: "))

    s = student(id, name, m1, m2, m3)

    Student[id] = s

    print(f"Added student: {s}")

    return True

```

```

def displayStudent():

    print("Student Details:")

    for sid, s in Student.items():

        print(f"Student ID : {s.get_sid()}")
        print(f"Name : {s.get_sname()}")
        print(f"M1 : {s.get_m1()}")
        print(f"M2 : {s.get_m2()}")
        print(f"M3 : {s.get_m3()}")
        print("-----")

    return True

```

```

def searchById(id):

    s = Student.get(id, -1)

    if s != -1:
        print(s)
        return True
    else:
        print("Id not found!")
        return False

```

```

def searchByName(name):

    found = False

    for s in Student.values():

        if s.get_sname() == name:
            print(s)
            found = True

    if not found:
        print("Name not found!")

    return found

```

```

def calculateGPA(id):

    s = Student.get(id, -1)

    if s != -1:
        print("GPA:", s.calculateGPA())

```

```
        return True
    else:
        print("Id not found!")
        return False
```

```
choice = -1
```

```
while choice != 0:
```

```
    choice = int(input("""
Enter Choice:
```

- 1. Add Student
- 2. Display All Students
- 3. Search by id
- 4. Search by name
- 5. Calculate GPA of a student
- 0. Exit

```
Enter Choice:
"""))
```

```
match choice:
```

```
    case 1:
        status = addStudent()
        print("Successfully Added") if status else print("Unsuccessful")
```

```
    case 2:
        displayStudent() if Student else print("No Students!")
```

```
    case 3:
        id = int(input("Enter Id: "))
        searchById(id)
```

```
    case 4:
        name = input("Enter name: ")
        searchByName(name)
```

```
    case 5:
        id = int(input("Enter Student Id: "))
        calculateGPA(id)
```

```
    case 0:
        print("Thanks")
```

```
    case _:
        print("Wrong Choice")
```

---

**Q2. Write a python program to store information of your friends  
id,name,lastname,hobbies,mobno,email,bdate,address  
note: hobbies- a friend may have multiple hobbies**

**Accept all friends details and store it in an array  
And do the following.**

**1. Display All Friend**

**2. Search by id**

**3. Search by name**

**4. Display all friend with a particular hobby**

**5. Exit**

class friend:

```
def __init__(self, fid=0, name="", lastname="",  
             hobbies=None, mobno="", email="",  
             bdate="", address=""):
```

```
    self.__fid = fid  
    self.__name = name  
    self.__lastname = lastname  
    self.__hobbies = hobbies if hobbies is not None else []  
    self.__mobno = mobno  
    self.__email = email  
    self.__bdate = bdate  
    self.__address = address
```

```
def __str__(self):  
    return f'fid : {self.__fid}, name : {self.__name}, lastname : {self.__lastname}, hobbies :  
{self.__hobbies}, mobno : {self.__mobno}, email : {self.__email}, bdate : {self.__bdate}, address :  
{self.__address}'
```

# Setters

```
def set_fid(self, fid):  
    self.__fid = fid
```

```
def set_name(self, name):  
    self.__name = name
```

```
def set_lastname(self, lastname):  
    self.__lastname = lastname
```

```
def set_hobbies(self, hobbies):  
    self.__hobbies = hobbies
```

```
def set_mobno(self, mobno):  
    self.__mobno = mobno
```

```
def set_email(self, email):  
    self.__email = email
```

```
def set_bdate(self, bdate):
    self.__bdate = bdate

def set_address(self, address):
    self.__address = address
```

# Getters

```
def get_fid(self):
    return self.__fid

def get_name(self):
    return self.__name

def get_lastname(self):
    return self.__lastname

def get_hobbies(self):
    return self.__hobbies

def get_mobno(self):
    return self.__mobno

def get_email(self):
    return self.__email

def get_bdate(self):
    return self.__bdate

def get_address(self):
    return self.__address
```

```
friends = [
```

```
    friend(
        fid=1,
        name="Alice",
        lastname="Smith",
        hobbies=["Reading", "Coding"],
        mobno="9876543210",
        email="alice@example.com",
        bdate="1995-04-12",
        address="123 Maple Street"
    ),

    friend(
        fid=2,
        name="Bob",
        lastname="Johnson",
        hobbies=["Gaming", "Hiking", "Cooking"],
```

```

        mobno="9123456789",
        email="bob@example.com",
        bdate="1998-09-25",
        address="456 Oak Avenue"
    ),

    friend(
        fid=3,
        name="Charlie",
        lastname="Brown",
        hobbies=["Photography", "Music"],
        mobno="9988776655",
        email="charlie@example.com",
        bdate="1992-11-05",
        address="789 Pine Road"
    )
]

```

```

class objectNotFound(Exception):

```

```

    def __init__(self, msg):
        self.__msg = msg

```

```

    def __str__(self):
        return self.__msg

```

```

def displayAllFriends():

```

```

    print("FRIENDS DETAILS....\n")

```

```

    for f in friends:

```

```

        print(f"Id : {f.get_fid()}")
        print(f"First Name : {f.get_name()}")
        print(f>Last Name : {f.get_lastname()}")
        print(f"Hobbies : {f.get_hobbies()}")
        print(f"Mobile No : {f.get_mobno()}")
        print(f>Email : {f.get_email()}")
        print(f"D.O.B. : {f.get_bdate()}")
        print(f"Address : {f.get_address()}")
        print("-----")

```

```

    return len(friends)

```

```

def searchById(fid):

```

```

    found = None

```

```

    try:

```

```
for ob in friends:
```

```
    if ob.get_fid() == fid:  
        found = ob  
        break
```

```
if found is None:
```

```
    raise objectNotFound("Not found: Friend ID does not exist.")
```

```
print(found)
```

```
return True
```

```
except objectNotFound as e:
```

```
    print(e)  
    return False
```

```
finally:
```

```
    print("--- Search execution finished ---")
```

```
def searchByName(name):
```

```
    found = None
```

```
    try:
```

```
        for ob in friends:
```

```
            if ob.get_name() == name:  
                found = ob  
                break
```

```
if found is None:
```

```
    raise objectNotFound("Not found: Friend name does not exist.")
```

```
print(found)
```

```
return True
```

```
except objectNotFound as e:
```

```
    print(e)  
    return False
```

```
finally:
```

```
    print("--- Search execution finished ---")
```

```

def searchByHobby(hobby):

    found = []

    try:

        for ob in friends:

            hobbies_list = ob.get_hobbies()

            if hobby in hobbies_list:
                print(ob)
                found.append(ob)

            if not found:
                raise objectNotFound(
                    "Not found: hobby does not exist for any friend."
                )

        return True

    except objectNotFound as e:

        print(e)
        return False

    finally:

        print("--- Search execution finished ---")

```

# Main

```
choice = -1
```

```
while choice != 0:
```

```
    try:
```

```
        print()
```

```
        choice = int(input("""
```

```
-----
Enter Choice:
```

1. Display All Friend
2. Search by id
3. Search by name
4. Display all friend with a particular hobby
0. Exit

Enter Choice:

```
"""))
```

```
print("-----")
```

except ValueError:

```
print("Please enter a valid number.")
continue
```

match choice:

case 1:

```
count = displayAllFriends()
```

if count:

```
print("These are all your friends :)")
```

else:

```
print("Not Found")
```

case 2:

```
fid = int(input("Enter the id to search: "))
searchById(fid)
```

case 3:

```
name = input("Enter the name to search: ")
searchByName(name)
```

case 4:

```
hobby = input("Enter hobby to search: ")
searchByHobby(hobby)
```

case 0:

```
print("Thanks!")
```

case \_:

```
print("Wrong Choice")
```

---

**2. Design a class hierarchy to maintain information for ABCTel telecom company , company wants to maintain information of customers and vendors.**

**For vendors they want to store vendorid, name, email, phone number,list of products they supply**

**For customers they want to store custid, name, email, credit class(based on credit history), discounts, plan assigned to customer**

**Customer may be Individual or a company**

**For individual customer, system should store phone number**

**For a customer of type Company, system should store info about relationship manager, credit line, extensions, list of numbers**

class Vendor:

```
def __init__(self, vendorid=0, name="", email="",
             phone="", products=None):

    self.__vendorid = vendorid
    self.__name = name
    self.__email = email
    self.__phone = phone
    self.__products = products if products is not None else []
```

# Getters

```
def get_vendorid(self):
    return self.__vendorid
```

```
def get_name(self):
    return self.__name
```

```
def get_email(self):
    return self.__email
```

```
def get_phone(self):
    return self.__phone
```

```
def get_products(self):
    return self.__products
```

# Setters

```
def set_vendorid(self, vendorid):
    self.__vendorid = vendorid
```

```
def set_name(self, name):
    self.__name = name
```

```
def set_email(self, email):
    self.__email = email
```

```
def set_phone(self, phone):
    self.__phone = phone
```

```
def set_products(self, products):
    self.__products = products if products is not None else []
```

```
def __str__(self):
    return f"VendorId: {self.__vendorid}, Name: {self.__name}, Email: {self.__email}, Phone
Number: {self.__phone}, Products: {self.__products}"
```

class Customer:

```
def __init__(self, custid=0, name="", email="",  
             credit_class="", discounts=0, plan=""):
```

```
    self.__custid = custid  
    self.__name = name  
    self.__email = email  
    self.__credit_class = credit_class  
    self.__discounts = discounts  
    self.__plan = plan
```

# Getters

```
def get_custid(self):  
    return self.__custid
```

```
def get_name(self):  
    return self.__name
```

```
def get_email(self):  
    return self.__email
```

```
def get_credit_class(self):  
    return self.__credit_class
```

```
def get_discounts(self):  
    return self.__discounts
```

```
def get_plan(self):  
    return self.__plan
```

# Setters

```
def set_custid(self, custid):  
    self.__custid = custid
```

```
def set_name(self, name):  
    self.__name = name
```

```
def set_email(self, email):  
    self.__email = email
```

```
def set_credit_class(self, credit_class):  
    self.__credit_class = credit_class
```

```
def set_discounts(self, discounts):  
    self.__discounts = discounts
```

```
def set_plan(self, plan):
```

```

    self.__plan = plan

    def __str__(self):
        return f'Customer Id: {self.__custid}, Name: {self.__name}, Email: {self.__email}, Credit
Class: {self.__credit_class}, Discounts: {self.__discounts}, Plan: {self.__plan}'

```

```

class Individual(Customer):

```

```

    def __init__(self, phone="", *t, **kwargs):

        super().__init__(*t, **kwargs)
        self.__phone = phone

    def get_phone(self):
        return self.__phone

    def set_phone(self, phone):
        self.__phone = phone

    def __str__(self):
        return super().__str__() + f', Phone: {self.__phone}'

```

```

class Company(Customer):

```

```

    def __init__(self, rmgr="", credit_line=0,
        extensions=0, numbers=None, *t, **kwargs):

        super().__init__(*t, **kwargs)

        self.__rmgr = rmgr
        self.__credit_line = credit_line
        self.__extensions = extensions
        self.__numbers = numbers if numbers is not None else []

    def get_rmgr(self):
        return self.__rmgr

    def set_rmgr(self, rmgr):
        self.__rmgr = rmgr

    def get_credit_line(self):
        return self.__credit_line

    def set_credit_line(self, credit_line):
        self.__credit_line = credit_line

    def get_extensions(self):
        return self.__extensions

    def set_extensions(self, extensions):

```

```
self.__extensions = extensions

def get_numbers(self):
    return self.__numbers

def set_numbers(self, numbers):
    self.__numbers = numbers

def __str__(self):
    return super().__str__() + f", Relationship Manager: {self.__rmgr}, Credit Line: {self.__credit_line}, Extensions: {self.__extensions}, Numbers: {self.__numbers}"
```